

Informe de Laboratorio 02

Tema: Desarrollo Orientado a Pruebas (TDD) y Comportamiento (BDD)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Pruebas de Software				
TÍTULO DE LA PRÁCTICA:	<i>Desarrollo Orientado a Pruebas (TDD) y Comportamiento (BDD)</i>				
NÚMERO DE PRÁCTICA:	02	AÑO LECTIVO:	2026	NRO. SEMESTRE:	VII
FECHA DE PRESENTACIÓN	24 de Abril 2026	HORA DE PRESENTACIÓN	12:00		
INTEGRANTE (s): • Chambilla Perca Ricardo • Alexandra Raquel Quispe • Paul Cari Lipe • Jhonatan David Quispe				NOTA:	
DOCENTE(s): <i>Prof. Robert Arisaca / Prof. Diego Iquira / Prof. Lino Pinto</i>					

1. EJERCICIOS/PROBLEMAS PROPUESTOS

- TDD: Validador de Identificadores (Basado en Myers).
- BDD: Proceso de Retiro en Cajero Automático (Caso ATM).
- Validador de Calificaciones.
- Calculadora de Reembolsos.

2. Desarrollo de los Ejercicios

Todos los códigos se encuentran ubicados en el directorio lab2/ del proyecto.

2.1. Ejercicio 1 - TDD: Validador de Identificadores (Basado en Myers)

Siguiendo estrictamente el flujo **Red-Green-Refactor** de TDD, implementaremos el validador de identificadores.

Paso 1: Fase RED (Escribir una prueba que falle)

Creamos el archivo lab2/test_identificador.py y probamos la condición básica (la longitud).

Listing 1: lab2/test_identificador.py (Fase RED)

```
1 import pytest
2 from identificador import es_identificador_valido
3
4 def test_longitud_valida():
5     # Longitud 1 (limite inferior)
6     assert es_identificador_valido("A") == True
```

Resultado esperado: La prueba falla con ImportError porque identificador.py no existe.

Paso 2: Fase GREEN (Código mínimo para pasar)

Creamos lab2/identificador.py y aplicamos la lógica mínima para que la prueba de longitud pase.

Listing 2: lab2/identificador.py (Fase GREEN)

```
1 def es_identificador_valido(identificador):
2     return len(identificador) >= 1 and len(identificador) <= 6
```

Resultado esperado: La prueba inicial ahora pasa a verde.

Paso 3: Fase REFACTOR (Corregir y optimizar)

Ajustamos la lógica para cumplir con la validación de letras al inicio y alfanuméricos después. Usaremos expresiones regulares para mayor limpieza.

Listing 3: lab2/identificador.py (Fase REFACTOR final)

```
1 import re
2
3 def es_identificador_valido(identificador):
4     # Refactorizacion: Validacion de entrada (Robustez)
5     if not isinstance(identificador, str):
6         return False
7
8     # Validacion limites Myers (1 a 6 caracteres)
9     if len(identificador) < 1 or len(identificador) > 6:
10         return False
11
12     # Regla: Primer caracter letra, resto letras o digitos
13     patron = r'^[A-Za-z][A-Za-z0-9]*$'
14     if not re.match(patron, identificador):
15         return False
16
17     return True
```

Paso 4: Validación final (Valores límite y equivalencia)

Actualizamos test_identificador.py con los casos de Myers solicitados.

Listing 4: lab2/test_identificador.py (Validación Final)

```
1 import pytest
2 from identificador import es_identificador_valido
3
4 def test_limites_longitud_myers():
5     assert es_identificador_valido("") == False # Longitud 0
6     assert es_identificador_valido("A") == True # Longitud 1
7     assert es_identificador_valido("Var123") == True # Longitud 6
8     assert es_identificador_valido("Var1234") == False # Longitud 7
9
10 def test_reglas_formato():
```

```
11 assert es_identificador_valido("1var") == False # Falla regla primer char
12 assert es_identificador_valido("var_1") == False # Falla char especial
13 assert es_identificador_valido("v@r") == False # Falla char especial
```

2.2. Ejercicio 2 - BDD: Proceso de Retiro en Cajero Automático (Caso ATM)

1. Fase BDD (Definición del Comportamiento)

Antes de programar, definimos el comportamiento usando el lenguaje **Gherkin**.

Listing 5: lab2/atm.feature

```
1 Feature: Proceso de Retiro en Cajero Automatico
2   Scenario: Intento de retiro sin fondos suficientes
3     Dado que mi saldo es 100
4     Cuando intento retirar 150
5     Entonces debo ver un mensaje de "Fondos Insuficientes"
```

2. Fase TDD (Ciclo de Desarrollo)

Traducimos los pasos de Gherkin a código Python. **Paso A (Red):** Creamos la prueba que llama a métodos inexistentes de una clase ATM.

Listing 6: lab2/test_atm.py (Fase RED)

```
1 import pytest
2 from atm import ATM
3
4 def test_retiro_fondos_insuficientes():
5     # Dado que mi saldo es 100
6     cajero = ATM(100)
7     # Cuando intento retirar 150
8     resultado = cajero.retirar(150)
9     # Entonces debo ver un mensaje de "Fondos Insuficientes"
10    assert resultado == "Fondos Insuficientes"
```

Paso B (Green & Refactor): Implementamos la lógica del simulador asegurando que bloquee la transacción.

Listing 7: lab2/atm.py (Fase GREEN/REFACTOR)

```
1 class ATM:
2     def __init__(self, saldo_inicial):
3         self.saldo = saldo_inicial
4
5     def retirar(self, monto):
6         if monto > self.saldo:
7             return "Fondos Insuficientes"
8         self.saldo -= monto
9         return "Retiro Exitoso"
```

Al ejecutar `pytest test_atm.py`, la prueba BDD/TDD pasa exitosamente, garantizando la robustez ante errores de usuario.

2.3. Ejercicio 3 - Validador de Calificaciones

Paso 1: Fase RED

Comenzamos probando la clase de equivalencia más baja en `lab2/test_calificaciones.py`.

Listing 8: `lab2/test_calificaciones.py` (Fase RED)

```
1 from calificaciones import evaluar_rendimiento
2
3 def test_calificacion_insuficiente():
4     assert evaluar_rendimiento(10) == "Insuficiente"
```

Paso 2 y 3: Fase GREEN y REFACTOR

Codificamos `lab2/calificaciones.py` manejando los rangos y lanzando la excepción `ValueError`.

Listing 9: `lab2/calificaciones.py` (Implementación Final)

```
1 def evaluar_rendimiento(nota):
2     if not isinstance(nota, int):
3         raise ValueError("La nota debe ser entera")
4
5     if nota < 0 or nota > 20:
6         raise ValueError("Nota fuera de rango")
7
8     if 0 <= nota <= 10:
9         return "Insuficiente"
10    elif 11 <= nota <= 15:
11        return "Regular"
12    else:
13        return "Excelente"
```

Paso 4: Validación final

Identificamos y probamos las clases de equivalencia válidas e inválidas.

Listing 10: `lab2/test_calificaciones.py` (Validación Final)

```
1 import pytest
2 from calificaciones import evaluar_rendimiento
3
4 def test_clases_validas():
5     # Fronteras 0 - 10
6     assert evaluar_rendimiento(0) == "Insuficiente"
7     assert evaluar_rendimiento(10) == "Insuficiente"
8     # Fronteras 11 - 15
9     assert evaluar_rendimiento(11) == "Regular"
10    assert evaluar_rendimiento(15) == "Regular"
11    # Fronteras 16 - 20
12    assert evaluar_rendimiento(16) == "Excelente"
13    assert evaluar_rendimiento(20) == "Excelente"
14
15 def test_clases_invalidas():
16     with pytest.raises(ValueError):
17         evaluar_rendimiento(-1)
18     with pytest.raises(ValueError):
19         evaluar_rendimiento(21)
```

2.4. Ejercicio 4 - Calculadora de Reembolsos

1. Diseño BDD (Gherkin)

Definimos el escenario de la regla especial VIP.

Listing 11: lab2/reembolso.feature

```
1 Feature: Cancelacion de reservas
2   Scenario: Cliente VIP cancela a ultima hora
3     Dado que un cliente es VIP
4     Y el monto de su reserva es 200
5     Cuando cancela la reserva con solo 2 horas de antelacion
6     Entonces recibe un 50% de reembolso equivalente a 100
```

2. Pruebas de Robustez y Límites (TDD - RED)

Planteamos los casos de prueba antes de la lógica.

Listing 12: lab2/test_reembolsos.py (Fase RED)

```
1 import pytest
2 from reembolsos import calcular_reembolso
3
4 def test_limites_reembolso():
5     assert calcular_reembolso(100, 72, False) == 50.0 # Limite exacto 72
6     assert calcular_reembolso(100, 24, False) == 50.0 # Limite exacto 24
7     assert calcular_reembolso(100, 23, False) == 0.0 # Debajo de 24
8
9 def test_escenario_vip_bdd():
10     assert calcular_reembolso(200, 2, True) == 100.0 # Escenario Gherkin
11
12 def test_proteccionErrores():
13     with pytest.raises(ValueError):
14         calcular_reembolso(-100, 48, False)
```

3. Implementación (TDD - GREEN/REFACTOR)

Aseguramos que la regla VIP tenga prioridad sobre la regla de última hora.

Listing 13: lab2/reembolsos.py (Fase GREEN/REFACTOR)

```
1 def calcular_reembolso(monto, horas, es_vip):
2     if monto < 0:
3         raise ValueError("El monto no puede ser negativo")
4
5     if horas > 72:
6         porcentaje = 1.0
7     elif 24 <= horas <= 72:
8         porcentaje = 0.5
9     else:
10        porcentaje = 0.0
11
12    # Prioridad Regla VIP
13    if es_vip and porcentaje < 0.5:
14        porcentaje = 0.5
15
16    return monto * porcentaje
```

3. Conclusiones

- El Desarrollo Orientado a Pruebas (TDD) garantiza una alta robustez del código mediante el ciclo Red-Green-Refactor, ya que obliga al desarrollador a considerar casos límite y validaciones de errores antes de escribir la lógica principal, logrando el modelo "Shift-Left" mencionado en la guía.
- El comportamiento BDD utilizando la sintaxis Gherkin (Dado-Cuando-Entonces) cierra la brecha de comunicación entre negocio y desarrollo. Al tener especificaciones ejecutables, las pruebas actúan como documentación viva del sistema.
- La aplicación del análisis de valores límite (Myers) mejora drásticamente la efectividad de las pruebas unitarias. Al enfocarse en las fronteras de las particiones (como se observó en los límites de horas 24 y 72, o la nota de 0 a 20), detectamos los "defectos off-by-one" (usar $>$ en vez de $>=$) con gran precisión.

4. CUESTIONARIO

1. ¿Por qué en TDD se considera que "la prueba es el primer cliente del código?" Se considera al test como el "primer cliente" porque, en TDD, la prueba interactúa con la función o módulo antes incluso de que la aplicación principal la utilice. Esto obliga al programador a diseñar la API desde la perspectiva del consumidor: pensar en qué parámetros recibe y cuán fácil de usar es, antes de enfocarse en los detalles de implementación interna.

2. Explique cómo el enfoque BDD ayuda a evitar errores de comunicación entre el Product Owner y el equipo de desarrollo. BDD emplea un "Lenguaje Ubicuo" (syntaxis Gherkin estructurada en Dado-Cuando-Entonces) que es comprensible tanto para personas con perfil de negocio (Product Owner) como para perfiles técnicos. Al definir los requerimientos como escenarios de prueba vivos, se elimina la ambigüedad; si el PO define un escenario y el código lo valida en verde, se garantiza que se construyó lo solicitado.

3. Según Myers, ¿qué es el .^a análisis de valor límite? por qué es más efectivo que probar solo valores aleatorios? El .^a análisis de Valor Límite.^{es} una técnica de diseño de pruebas basada en la premisa de que los errores (bugs) tienden a concentrarse en las fronteras de las clases de equivalencia (por ejemplo, el límite inferior y superior de un rango). Probar exactamente el límite y el salto de frontera es más efectivo que usar valores aleatorios porque ataca directamente los típicos errores de lógica condicional matemática.

4. ¿Cuál es el propósito del paso Refactor en el ciclo de TDD y qué riesgo se corre si se omite? El propósito del paso *Refactor* es limpiar, optimizar y mejorar la legibilidad y estructura del código escrito en la fase *Green*, sin alterar el comportamiento externo. Si se omite, el código acumulará "Deuda Técnica", volviéndose progresivamente caótico, difícil de mantener e inescalable a largo plazo.

5. Explique la importancia de definir el resultado esperado antes de la ejecución de una prueba. Definir el resultado esperado previamente evita el "sesgo de confirmación". Si ejecutamos el código primero y vemos su salida, tendemos a justificar que "parece correcto" adaptamos la prueba al comportamiento actual del código. Definirlo antes garantiza que validemos la funcionalidad contra las reglas de negocio reales.

5. Referencias y Bibliografía

- 1 Andreas Spillner, "Software Testing Foundations", Editorial Rocky Nook, 2021, 5th Edition.
- 2 Glenford J. Myers, "The Art of Software Testing", Editorial Jhon Wiley & Sons, 2012, 3th Edition.
- 3 Documentación oficial de Pytest: <https://docs.pytest.org/en/stable/>